



MUCHUO'S CENTRE FOR TRAINING & DEVELOPMENT

Complete Software Testing Programme

"From Beginner to Job Ready"

INSTRUCTOR GUIDE

Facilitation · Labs · ISTQB Fridays · Career Coaching

How to use this guide

This handbook is the teaching companion to the Student Programme Guide, the Complete Course slide deck, and the Presenters Guide. Use it session-by-session from introduction through practicals and CV coaching.

Companion materials

- **Student Programme Guide** — share with learners (journey, tools, outcomes).
- **Course Preview** — learner-facing slides.
- **Presenters Guide** — same slides with bottom presenter notes (theory spine, slides 1–144).
- **This Instructor Guide** — pacing, lab runbooks, ISTQB Fridays, CV coaching, sign-off.

How each phase page is organised

- **Intent** — what learners must leave knowing or able to do.
- **Source slides / notes** — where to teach from.
- **Session flow** — teach → demo → practice → check.
- **Talking points** — compressed summary from the course manual & notes.
- **Lab / homework** — practical evidence to collect.
- **Pitfalls** — common learner blockers.

Delivery model: 12–16 weeks · ~2 hrs daily self-study · Sat–Sun live (2–4 hrs) · Friday ISTQB classes · online tutor support. Theory from the slide deck first; practicals (Postman, Azure DevOps, Visual Studio frameworks) once STLC and techniques are secure.

Programme map for instructors

Align every live class to the nine-phase student journey. Theory slides currently cover Phase 1 deeply; Phases 2–9 are delivered as instructor-led labs using this guide.

Phase	Focus	Primary teaching source	Live emphasis
1	Fundamentals	Slides 1–144 + presenter notes	Concepts, vocabulary, ISTQB hooks
2	Manual UI	STLC slides + this lab runbook	Azure DevOps: story → case → bug → retest
3	Manual API	Layers / networking slides + lab	Postman & Swagger on REST APIs
4	ISTQB CTFL	ISTQB summary notes (this guide) + Friday mocks	Parallel with Phases 2–3
5	SQL	DB / SQL slides + exercises	Validate data behind UI/API
6	Automation UI & API	Practical runbooks	VS + NuGet: Selenium / Reqnroll / RestSharp
7	Mobile	Overview + demo lab	Maestro / Appium orientation
8	CI/CD	Pipeline lab	ADO / GitHub Actions running tests
9	Career	CV coaching pages	CV, LinkedIn, mocks, portfolio

Instructor prep checklist

Accounts & access (before week 1)

- Microsoft / Azure DevOps org + sample project with User Stories
- Postman workspace (team or classroom) + sample public API (e.g. ReqRes / Swagger Petstore)
- GitHub org or classroom repos for UI & API automation starter solutions
- Shared drive for evidence: screenshots, exports, CVs
- ISTQB CTFL syllabus PDF + mock exam packs for Fridays

Software learners must install

- **Visual Studio** (Community) with .NET desktop / ASP.NET workloads
- Git + GitHub Desktop or CLI
- Postman desktop
- Chrome / Edge (for Selenium WebDriver)
- Optional: Rider, Android Studio / emulator for mobile phase

NuGet packages (Phase 6 starter kits)

UI: Selenium.WebDriver · Selenium.Support · DotNetSeleniumExtras.WaitHelpers · Reqroll / SpecFlow · NUnit or xUnit · FluentAssertions · ExtentReports or similar

API: RestSharp · Newtonsoft.Json or System.Text.Json · NUnit/xUnit · FluentAssertions · Reqroll (optional BDD)

Pre-build two Visual Studio solution templates (UI Automation Framework + API Automation Framework) so live time is spent on design patterns, not package restore fights.

Opening sessions — Intro & mindset

Slides 1–6

Live: 45–60 min

Outcome: trust, commitment, map of the journey

Intent

Introduce the instructor, set belief/mindset, and walk the course outline so learners see theory → Postman → Azure DevOps → VS automation → ISTQB → CV.

Session flow

1. Title & instructor intro (Slide 1–2).
2. Belief System: fear → understanding; consistency (Slides 3–4).
3. Outline part A & B (Slides 5–6) — pause on practicals and career.
4. Show Student Programme Guide Phases 1–9 on screen.

Talking points (summary)

- Non-IT backgrounds are expected; testing is learned step by step.
- Everyone in tech started at zero; commitment beats talent myths.
- Outline promises: foundations, manual UI/API, Postman/Swagger, ADO test management, Selenium/RestSharp/BDD, CI/CD, ISTQB + CV.

Pitfall: Do not deep-dive tools on day one. Plant the destination, then earn it through fundamentals.

Phase 1A — Systems foundations

Slides 7–18

1–2 live blocks

Computer, binary, hardware/software, networks

Intent

Give non-technical learners a shared vocabulary: computers, binary, software types, client-server communication. This underpins later API and environment talks.

Checks for understanding

- Name hardware vs software with one everyday example each.
- Explain (simply) how a phone reaches a server.
- Why testers care about networks/APIs (failure location).

Summary notes for teaching

- Binary is how machines represent data — testers meet it as encodings, flags, and “odd” stored values.
- Software types (system vs application) frame what we test.
- Client → network → server is the mental model for UI vs API failures (presenter note on Slide 17).
- Use the network-devices diagram (Slide 18) as a slow walkthrough, not a rush.

ISTQB hook: Plant the idea that defects can sit in interface, logic, or data — revisited on objectives / layers slides.

Phase 1B — Waterfall & requirements

Slides 19–44

1–2 live blocks

BRD / FRS, design, build, test, deploy, maintain

Session flow

1. Methodologies list — focus Waterfall, V-Model, Agile (Slide 20 note).
2. Waterfall cascade — same stages exist in every method; order/overlap change.
3. BRD vs FRS with login / e-commerce examples from presenter notes.
4. Design → development → system test → deploy (Azure focus) → ops.
5. Close on Waterfall advantages vs challenges → why Agile emerged.

Summary notes

- **BRD** = business view; **FRS** = testable detail (fields, rules, messages).
- Testing should start when documents exist — not only at “System Test” box (Slide 36 note).
- Cite real failure cost (e.g. Knight Capital) for “why testing matters”.
- Cloud for this course = Microsoft Azure.

Homework

Learners write 5 BRD-level bullets and 5 FRS-level bullets for a “Forgot password” feature.

Phase 1C — Agile & Scrum

Slides 45–62

1–2 live blocks

Roles, backlog, sprint ceremonies

Intent

Learners can describe Epic → Story → Task, name ceremonies, and explain continuous testing inside the sprint.

Talking points

- Agile keeps docs practical; delivery + feedback over late big-bang releases.
- PO translates user need; Scrum Master removes blockers; Dev team includes testers.
- Flow: Product backlog → Sprint backlog → ceremonies → tested increment.
- Key takeaway (Slide 62): testing is continuous, not end-loaded.

Activity (15 min)

In pairs: turn “Online checkout” into 1 Epic, 3 Stories, 2 testing Tasks. Peer critique for testability.

Bridge forward: these Stories become Azure DevOps work items in Phase 2.

Phase 1D — V-Model

Slides 63–72

~1 live block

Verification & validation pairing

Summary notes

- Each left-side development stage pairs with a right-side test level.
- Requirements ↔ Acceptance; System design ↔ System test; Architecture ↔ Integration; Module design ↔ Unit.
- Coding sits at the bottom; execution climbs the right side.
- Strength: early test design. Challenge: change cost if requirements shift late.

Teach tip

Draw the V live once. Ask: “If the FRS is wrong, which test level fails first in production impact?” Guide them to acceptance / system, then show how earlier reviews prevent that.

ISTQB: V-Model questions often test pairing of levels — drill the pairs every Friday until automatic.

Phase 1E — Testing intro & layers

Slides 73–81

~1 live block

Definition, objectives, UI / server / DB / SQL

Summary notes (course manual)

- Testing compares actual behaviour to requirements/stories/designs and produces evidence.
- Find defects → raise clearly → retest until gone.
- Main objective: software meets user requirements across **UI**, **servers**, and **data**.
- SQL is how we inspect and validate stored data — preview of Phase 5.

Demo

Open any web app. Point to UI. Open DevTools Network for server/API. Optionally show a table in a DB tool. Say: “Your career will move between these three layers.”

Check

Learners classify three bug examples as UI, server/API, or data.

Phase 1F — STLC (plan → close)

Slides 82–112

2–3 live blocks

Core manual-testing professional practice

Session flow (recommended split)

1. **Block A:** Six STLC phases diagram → Requirement analysis → Test planning (plan contents, RACI, scope, approach, entry/exit, suspension, deliverables, schedule, risks).
2. **Block B:** Scenarios → Cases → Scripts → Expected results → RTM.
3. **Block C:** Environments (Dev → Live) → Execution Pass/Fail → Bug workflow → Retest / UAT defects → Closure.

Summary notes

- One scenario → many cases; cases need data + expected results.
- RTM proves coverage: requirement ↔ cases.
- Bug loop: log → triage → fix → retest → closed (same in Jira or Azure DevOps).
- Evidence beats opinion: steps, data, screenshots, environment.

Bridge to Phase 2

Promise: “Next we will do this end-to-end in Azure DevOps — User Story → Test Case → Bug → Retest.”
Keep vocabulary identical so the tool feels like paperwork they already understand.

Phase 1G — Techniques & reviews

Slides 113–133

2 live blocks

Black-box techniques + review types

Summary notes

- Four families: Reviews, Non-functional, White-box, Black-box.
- Black-box (QA daily): positive/negative, BVA, EP, state transition, error guessing, exploratory, smoke, retesting.
- BVA: edges and just-inside/outside; EP: one/few from each class.
- Reviews: walkthrough (author leads, many roles), peer (testers), technical (architects/devs), informal (quick draft skim).

Classroom drills

- Age rule: derive EP classes + BVA ages (Slides 121–123).
- ATM / login state diagram: valid vs illegal transitions.
- Quiz: who leads walkthrough vs peer vs technical (ISTQB favourite).

Interview trap: Mixing retesting vs regression — fix vocabulary before Phase 1H closes.

Phase 1H — Non-functional & maintenance

Slides 134–144

~1 live block

Quality attributes + regression vocabulary

Summary notes

- Functional = does it match the spec? Non-functional = how well (secure, fast, stable, usable, scalable).
- Performance family: smoke (light), load, stress, spike, endurance/soak.
- Maintenance testing is risk-based after production change.
- **Retest** = same failing case after fix. **Regression** = wider set for side effects.

Close Phase 1

Celebrate vocabulary mastery. Preview practical arc: Azure DevOps UI testing → Postman API → SQL → Visual Studio automation frameworks → CI/CD → CV.

Slide 144 “The End” is end of theory deck — not end of programme.

Phase 2 — Manual UI testing with Azure DevOps

Practical lab

2–3 live weekends

User Story → Test Case → Bug → Retest

Lab objective

Every learner executes a small UI feature end-to-end in Azure DevOps and produces portfolio evidence (exported cases, bug with attachments, retest result).

Instructor demo (30–40 min)

1. Open ADO project board / Backlog.
2. Create/refine a User Story with acceptance criteria.
3. Create Test Plan / Suite / Cases linked to the story.
4. Run a test → Fail → create Bug with steps, expected vs actual, severity, evidence.
5. Simulate fix → Retest → mark Passed / close bug.

Learner practice

- App under test: agreed training site (or instructor-hosted demo app).
- Minimum: 1 story, 8–12 cases (positive + negative + BVA where relevant), 1 bug, 1 retest.
- Capture screenshots in ticket and in personal portfolio folder.

Sign-off criteria

- Cases are requirement-backed and atomic.
- Bug is reproducible by a peer without asking questions.
- Learner can narrate the workflow without reading notes.

Teaching link: Reuse STLC language from slides 96–110 so ADO feels familiar. Mention Jira parity (fields differ; process does not).

Phase 3 — Manual API testing with Postman

Practical lab

2 live weekends

Postman + Swagger / OpenAPI

Session flow

1. Recap client-server & JSON from foundations.
2. Open Swagger/OpenAPI docs — identify resources, verbs, schemas.
3. Postman: collections, environments, variables, auth (API key / Bearer).
4. Send GET/POST/PUT/PATCH/DELETE; read status codes & body.
5. Write test scripts (status, JSON property asserts) — still “manual API” mindset with light scripting.
6. Negative tests: bad auth, missing fields, wrong IDs.

Portfolio evidence

- Exported Postman Collection + Environment.
- Short README: base URL, auth setup, 10+ requests covering CRUD + negatives.
- 2–3 screenshots of failing vs passing assertions.

Pitfalls

- Learners paste tokens into screenshots — teach redaction.
- Treating 200 as always success — assert body too.
- Skipping Swagger and guessing URLs.

Phase 4 — ISTQB Foundation (Friday classes)

Parallel with Phases 2–3

Every Friday

CTFL syllabus • mocks • technique

How to run Fridays

45–90 minutes. Rotate: (1) syllabus teach-back of one chapter, (2) 10–15 MCQs timed, (3) review wrong answers with “why the distractor tempts you”, (4) vocabulary flash (retest vs regression, review types, V-pairs, BVA/EP).

ISTQB summary map (teach from course notes)

- **Fundamentals:** why test, seven principles, test process.
- **Lifecycle:** levels & types; V-Model & Agile testing fit.
- **Static testing:** reviews & roles (walkthrough / peer / technical / informal).
- **Techniques:** black-box (EP, BVA, decision, state), experience-based.
- **Management:** planning, risk, estimation, defects, tools overview.

Link to our deck

- Objectives & layers → principles & object of testing.
- STLC / plan contents → test management questions.
- Techniques slides 113–133 → largest exam scoring area.
- Presenter notes already flag exam angles — reuse them aloud.

Target: learners sit a full mock before Phase 6; aim comfortable pass margin before programme end.

Phase 5 — SQL for testers

Slides 80–81 + exercises

1–2 sessions

Data validation behind UI/API

Teach & practice

- SELECT / WHERE / ORDER BY
- INNER vs LEFT JOIN
- GROUP BY / COUNT for smoke data checks
- Compare UI/API response fields to DB rows

Use a tiny sample schema (Users, Orders) so joins are visible.

Evidence

- Exercise sheet with 8 queries + expected result screenshots.
- One end-to-end: create via API/UI → SELECT proves persistence.

Phase 6A — UI automation framework (Visual Studio)

Practical lab

2–3 weekends

Selenium · Reqnroll/BDD · NuGet

Framework shape to teach

Solution folders: *Tests* (JUnit/xUnit or Reqnroll features) · *Pages* (Page Object Model) · *Core* (Driver factory, waits, config) · *Utilities* (reporting, screenshots). NuGet: Selenium.WebDriver, Selenium.Support, wait helpers, Reqnroll, assertions, reporting.

Live build sequence

1. Create NUnit/xUnit test project in Visual Studio.
2. Install NuGet packages; restore; tip: commit `packages.lock` / use `PackageReference`.
3. Driver factory (Chrome) + quit in teardown.
4. First raw test (login or search) — then refactor to Page Objects.
5. Add Reqnroll feature file (Given/When/Then) binding to steps.
6. Add screenshot on failure + simple HTML/Extent report.

Learner outcomes

- Explain POM and why selectors live in pages.
- Run tests from Test Explorer and command line (`dotnet test`).
- At least 5 scenarios automated from earlier manual cases.

Pitfalls

- Brittle absolute XPaths — teach relative + waits.
- No waits → flaky demos live.
- Secrets in repo — use user secrets / env vars.

Phase 6B — API automation framework (Visual Studio)

Practical lab

1–2 weekends

RestSharp · NuGet · assertions

Live build sequence

1. Create test project; install RestSharp + JSON + NUnit/xUnit + FluentAssertions.
2. Base client wrapper (base URL, auth header, default headers).
3. Model classes for request/response DTOs.
4. Tests: GET list, GET by id, POST create, PUT/PATCH update, DELETE — assert status + payload.
5. Optional: Reqnroll scenarios reusing Postman collection cases.
6. Optional: read config from `appsettings.json` / environment.

Portfolio evidence

- GitHub repo with README (how to run, packages used).
- Same business flow covered manually in Postman now automated.
- Screenshot of Test Explorer green run.

Narrative for interviews: “I designed a C# API framework with RestSharp, covered CRUD + negatives, and mirrored cases from my Postman suite.”

Phases 7–8 — Mobile & CI/CD

Orientation + pipeline lab

1–2 weekends

Maestro/Appium · ADO / GitHub Actions

Phase 7 — Mobile (overview depth)

- Show emulator vs real device trade-offs.
- Demo one Maestro flow or Appium smoke on Android.
- Learners document differences vs web automation (locators, gestures, resets).
- Stretch: clone starter mobile framework into portfolio.

Phase 8 — CI/CD

1. Push UI or API automation repo to GitHub.
2. Add GitHub Action or Azure Pipeline: restore → `dotnet test` → publish results.
3. Break a test on purpose; show red build; fix; show green.
4. Discuss gates: when fail should block merge/release.

Tie back to smoke vs regression: pipelines often run smoke every PR and wider suites nightly.

Phase 9 — CV, LinkedIn & interviews

Career preparation

Final 1–2 weeks (overlap earlier)

Job-ready packaging

CV coaching

- One page (or two max): profile, skills, projects, employment/education.
- Projects named as outcomes: “Postman suite — 25 requests, auth, negatives”; “Selenium POM + Reqroll — 12 scenarios”; “RestSharp API framework”; “ADO test management sample”.
- Ban empty claims (“team player”) without evidence.
- Instructor reviews each CV once with tracked comments.

LinkedIn & portfolio

- Headline: Software Tester / QA · Manual + API + Automation.
- Featured: GitHub repos + PDF Student Guide journey optional.
- About section mirrors CV projects in first person.
- Screenshot evidence folders cleaned and linked.

Interview coaching

- Drill: STLC, bug life cycle, BVA/EP, retest vs regression, Agile tester role.
- Tool stories: narrate ADO and Postman workflows aloud.
- Framework walkthrough: open VS, explain folders, run one test.
- At least one timed mock interview per learner.

Assessment & programme sign-off

Use this checklist before marking a learner “programme complete / job-ready packaging done”.

1 Explains SDLC/STLC, Agile ceremonies, V-pairs without slides

2 Designs cases with EP/BVA/negative coverage

3 Azure DevOps: story → cases → bug → retest evidence

4 Postman collection + Swagger-based API tests

5 SQL checks validating UI/API data

6 UI automation framework (VS + Selenium + NuGet) runs cleanly

7 API automation framework (VS + RestSharp + NuGet) runs cleanly

8 CI pipeline executes automated tests

9 ISTQB mock at target score / exam plan set

10 CV + LinkedIn + portfolio reviewed; mock interview done

Weekend session planner (template)

Copy this template each week. Keep theory blocks short once practicals start; protect lab time.

Block	Saturday	Sunday	Friday ISTQB
0–15 min	Recap + vocabulary quiz	Homework review	Warm-up MCQs
15–60 min	Teach / demo (slides or tool)	Continue lab build	Syllabus focus topic
60–120+ min	Guided practice	Independent practice + evidence upload	Timed quiz + review
Close	Assign homework	Sign-off / blockers	Flashcards for weak areas

Instructor habit: After every live class, note who is blocked on installs, auth tokens, or flaky tests — resolve before next weekend so momentum holds.

Quick reference — materials & ownership

Document ownership

- **Student Programme Guide** — marketing + learner orientation (8 pages).
- **Course Preview / Presenters Guide** — theory spine & presenter notes.
- **Instructor Guide (this file)** — facilitation from intro through practicals and CV.
- **Starter VS solutions** — maintain separately in GitHub; link from README.

Suggested next improvements

- Extend slide deck beyond 144 with Postman / ADO / automation slide modules.
- Attach printable ISTQB flashcards PDF.
- Record short lab demos for catch-up learners.

Closing note for instructors

Teach vocabulary early, tools second, frameworks third, and career packaging always with evidence. The programme promise is “Beginner to Job Ready” — your job is to leave no phase without a tangible artefact in the learner’s portfolio.

Appendix A — ISTQB summary notes (1/2)

Condensed teach-from notes drawn from the course manual, presenter notes, and CTFL-aligned vocabulary. Use on Fridays and as revision handouts.

Fundamentals

- Testing shows presence of defects, not their absence.
- Exhaustive testing is impossible — use risk, priorities, techniques.
- Early testing saves cost; defects cluster; pesticide paradox → refresh tests.
- Testing is context-dependent; absence-of-errors fallacy (right product still matters).
- Object of testing: compare behaviour to requirements; produce evidence.

Test levels (V-pairs)

- Unit ↔ module design
- Integration ↔ architecture
- System ↔ system design
- Acceptance ↔ requirements

Static testing / reviews

- **Walkthrough:** author/BA/PO presents; many roles ask questions.
- **Peer review:** testers review testware.
- **Technical review:** architects/senior devs lead design critique.
- **Informal:** quick two-person draft skim, light process.

Black-box techniques

- **EP:** classes of inputs with same expected behaviour; pick representatives.
- **BVA:** on and just beyond boundaries (inclusive/exclusive per spec).
- **State transition:** valid sequences vs illegal shortcuts.
- **Decision / cause-effect:** combinations of conditions → outcomes.
- **Experience-based:** error guessing + exploratory (chartered).

Appendix A — ISTQB summary notes (2/2)

Types & vocabulary traps

- **Smoke (functional):** build sanity — “does it start?”
- **Retesting:** same failing case after a fix.
- **Regression:** wider set for side effects of a change.
- **Non-functional:** security, reliability, usability, performance (load/stress/spike/soak), availability, recovery.
- White-box: structure visible (mostly developer unit/CI). Black-box: behaviour from outside (manual or automated).

Management snippets

- Entry/exit criteria gate phases.
- Risk-based testing directs depth.
- Defect report: title, steps, expected vs actual, environment, evidence, severity/priority.
- Traceability (RTM): requirements ↔ cases ↔ defects.

Friday drill set (rotate)

- 10 MCQs timed (12–15 minutes).
- One worked BVA/EP age or PIN example on whiteboard.
- Two “who participates?” review questions.
- One retest vs regression scenario story.
- Learners write one wrong-answer explanation in their own words.

Pass strategy: vocabulary first, then techniques maths (BVA/EP), then process/management. Weak English readers: practise slow reading of stems — many errors are misread qualifiers (always / only / most).

Appendix B — Practical summary notes

Azure DevOps (UI)

- Story with clear acceptance criteria.
- Test Plan → Suite → Cases linked to story.
- Run → Pass/Fail with comment + attachment.
- Bug from failed run; severity; repro steps.
- Retest after fix; close when evidence shows pass.

Postman (API)

- Start from Swagger paths & schemas.
- Environments for baseUrl + tokens.
- CRUD + auth + negatives.
- Assert status *and* body fields.
- Export collection for portfolio.

VS automation

- **UI:** Selenium + POM + waits + Reqnroll + reporter.
- **API:** RestSharp client wrapper + DTOs + FluentAssertions.
- NuGet PackageReference; run via Test Explorer / `dotnet test`.
- CI: restore → test → publish results.
- CV line = stack + count of scenarios + repo link.

Evidence standard (all practicals)

If a peer cannot reproduce the learner's result from the artefact alone (repo README, ADO links/screenshots, Postman export), it is not yet portfolio-ready. Coach specificity over screenshots of green ticks with no context.